



创新创业实践

Project5

说明文档

郎泓森

202200460010

一、SM2 基础实现及优化

1. 核心类解析

EllipticCurve（原始仿射坐标实现）

核心方法：

point_addition：实现仿射坐标系下的点加法

point_multiply：基于倍点算法的点乘法（核心运算）

inv_mod：扩展欧几里得算法求模逆（运算瓶颈）

特点：

每次点加/倍点都需要计算模逆（耗时操作）

实现直观但效率较低

EllipticCurveJacobi（优化雅可比坐标实现）

核心方法：

jacobian_point_double：雅可比坐标下的点加倍运算

jacobian_point_add：雅可比坐标下的点加运算

point_multiply：重写点乘法（使用坐标转换）

优化原理：

使用三元组 (x, y, z) 表示点

延迟模逆计算到坐标转换阶段

避免重复模逆运算（主要性能瓶颈）

SM2（国密算法实现）

参数：使用 SM2 标准曲线参数

P：256 位素数域

A/B：曲线系数

N：基点阶

G：(Gx, Gy) 基点坐标

核心功能：

generate_keypair: 生成公私钥对

sign: 消息签名

verify: 签名验证

2. 算法关键流程

密钥生成:

```
d = random.randint(1, N-1)          # 私钥 (随机整数)
P = curve.point_multiply(d, G)       # 公钥 (椭圆曲线点)
```

签名过程:

```
k = random.randint(1, N-1)          # 随机数
(x1, _) = point_multiply(k, G)      # 计算 k*G
e = sm3_hash(message)               # 消息哈希
r = (e + x1) % N                    # 第一部分签名
s = inv(1+d) * (k - r*d) % N        # 第二部分签名
```

验证过程:

```
t = (r + s) % N                     # 中间参数
sG = point_multiply(s, G)           # s倍基点
tP = point_multiply(t, public_key)  # t倍公钥
(x1, _) = point_addition(sG, tP)    # 椭圆曲线点加
R = (e + x1) % N                     # 验证计算值
return R == r                       # 验证结果
```

3. 性能对比实现

```
# 测试原始版本
orig_gen, orig_sign, orig_verify = test_sm2(use_jacobi=False)

# 测试优化版本
opt_gen, opt_sign, opt_verify = test_sm2(use_jacobi=True)

# 性能对比输出
print(f"密钥生成加速: {orig_gen/opt_gen:.2f}x")
print(f"签名加速: {orig_sign/opt_sign:.2f}x")
print(f"验证加速: {orig_verify/opt_verify:.2f}x")
```

4. 雅可比坐标优化原理

操作	仿射坐标计算代价	雅可比坐标计算代价
点加倍 (2P)	1 次求逆 + 2 次乘法	4 次乘法
点加 (P+Q)	1 次求逆 + 3 次乘法	12 次乘法
点乘 (k·P)	~512 次求逆 (256 位密钥)	全程仅 1 次求逆（坐标转换）

优化核心：将昂贵的模逆运算次数从 $O(\log k)$ 降低到 $O(1)$

5.运行结果

```

=====
测试原始版本（仿射坐标）
原始版（仿射坐标）生成密钥对...
密钥生成时间：21.1892 ms
原始消息：Hello, SM2 digital signature!

原始版（仿射坐标）生成签名...
签名结果：r=0x80357dc7838200a3a0111c621f405265d77fc1983861f27a9330443d944aa7f, s=0x4132da0e875b7b37726523c98f6d629080d90529f52b62273d02d2098d0d5bb4
签名时间：21.9166 ms

原始版（仿射坐标）验证签名...
验证结果：成功
验证时间：42.2747 ms


测试优化版本（雅可比坐标）
优化版（雅可比坐标）生成密钥对...
密钥生成时间：2.0053 ms
原始消息：Hello, SM2 digital signature!

优化版（雅可比坐标）生成签名...
签名结果：r=0x24426febd1432b4a35a2fdaacbb764c02a2b4375e968b4b6d7913c76133dd4a, s=0x2873290925a3707d4f9c5ad55e155a6b8f8a643b06e1a7371642d48d93a7a
签名时间：1.9987 ms

优化版（雅可比坐标）验证签名...
验证结果：成功
验证时间：5.4212 ms


=====
性能对比：
密钥生成加速：10.57x (21.19 ms -> 2.01 ms)
签名加速：10.97x (21.92 ms -> 2.00 ms)
验证加速：7.80x (42.27 ms -> 5.42 ms)

```

根据结果显示，原始版和优化版签名均通过了正确性验证，优化后的算法效率提高了将近 7 倍。

二、SM2 签名误用及分析

数学推导表示

1. 密钥泄漏攻击（Leaking k）

推导过程：

由 SM2 签名方程：

$$s = (1 + d_A)^{-1} \cdot (k - r \cdot d_A)$$

直接解方程：

$$s(1 + d_A) = k - r d_A$$

整理得：

$$d_A = \frac{k - s}{s + r} \bmod n$$

POC 验证：

安全防护建议

1. 随机数生成：

使用 RFC6979 确定性随机数生成

采用硬件安全模块(HSM)生成随机数

2. 密钥管理：

3. 算法选择：

实现加固：

添加随机数审计日志

实现签名计数器和阈值机制

采用两方签名方案（2P-Sign）

2. 相同用户重用随机数 k 攻击

数学推导：

$$\begin{cases} s_1 = (1 + d_A)^{-1} \cdot (k - r_1 \cdot d_A) \bmod n \\ s_2 = (1 + d_A)^{-1} \cdot (k - r_2 \cdot d_A) \bmod n \end{cases}$$

消去公因子：

$$\begin{aligned} s_1(1 + d_A) &= k - r_1 d_A \\ s_2(1 + d_A) &= k - r_2 d_A \end{aligned}$$

联立方程：

$$(s_1 - s_2)(1 + d_A) = (r_2 - r_1)d_A$$

最终推导私钥公式：

$$d_A = \frac{s_2 - s_1}{s_1 - s_2 + r_1 - r_2} \bmod n$$

POC 验证步骤:

1. 生成固定 k 值
2. 对两条不同消息 $M1, M2$ 签名
3. 提取 $(r1, s1)$ 和 $(r2, s2)$
4. 用推导公式计算私钥 dA
5. 与真实私钥比对

3. 不同用户重用随机数 k 攻击

数学推导:

$$\begin{cases} s_A = (1 + d_A)^{-1} \cdot (k - r_A \cdot d_A) \\ s_B = (1 + d_B)^{-1} \cdot (k - r_B \cdot d_B) \end{cases}$$

分析方程组:

包含两个未知数 dA, dB

仅有一个共享参数 k

无法构造有效方程组求解

验证结论:

pitfalls	ECDSA	Schnorr	SM2-sig
Leaking k leads to leaking of d	✓	✓	✓
Reusing k leads to leaking of d	✓	✓	✓
Two users, using k leads to leaking of d , that is they can deduce each other's d	✓ RFC 6979	✓ RFC 6979	✓
Malleability, e.g. (r, s) and $(r, -s)$ are both valid signatures, lead to blockchain network split	✓	✓	$r = (e + x_1) \bmod n$ $e = \text{Hash}(Z_A M)$
Ambiguity of DER encode could lead to blockchain network split	✓	✓	----
One can forge signature if the verification does not check m	✓	✓	✓
Same d and k with ECDSA, leads to leaking of d	✓	✓	✓

4. 签名可锻性攻击

合法签名 (r, s) 可被篡改为 $(r, -s \bmod n)$, 但验证结果依赖实现逻辑。

原始签名验证方程:

$$t \equiv r + s \pmod{n}, R \equiv e + x_1 \pmod{n}$$

其中 $(x_1, y_1) = sG + tP$

变体签名 (r, s') ($s' = -s$)

$$t' \equiv r + s' \equiv r - s \pmod{n}$$

验证点计算

$$(x'_1, y'_1) = s'G + t'P = -sG + (r - s)P$$

无法推导到 kG 。

5. 不检查消息的伪造攻击

系统仅验证 (r, s) 符合曲线数学关系，但未关联具体消息 M

攻击者随机生成 $r, s \in [1, n - 1]$ ，构造签名 (r, s)

若验证流程为计算 $e = SM3(Z_A || M)$ ，跳过 e 的计算和比较 $R = r$ 的步骤。系统错误接受未绑定消息的签名。

6. SM2 与 ECDSA 重用随机数 k 攻击

数学推导：

$$\begin{cases} ECDSA: s_1 = k^{-1}(e_1 + r_1d) \pmod{n} \\ SM2: s_2 = (1 + d)^{-1}(k - r_2d) \pmod{n} \end{cases}$$

变形方程组：

$$\begin{cases} k = s_1^{-1}(e_1 + r_1d) \\ k = s_2(1 + d) + r_2d \end{cases}$$

消去 k ：

$$s_1^{-1}(e_1 + r_1d) = s_2 + (s_2 + r_2)d$$

最终解：

$$d = \frac{s_1s_2 - e_1}{r_1 - s_1s_2 - s_1r_2} \pmod{n}$$

POC 验证流程：

1. 生成共享随机数 k
2. ECDSA 签名生成 (r_1, s_1)
3. SM2 签名生成 (r_2, s_2)

4. 通过公式推导私钥 d

代码解释

一、基础椭圆曲线实现

ECPoint 类

表示椭圆曲线上的点，包含坐标(x, y)。支持点相等性判断和格式化输出（如无穷远点处理）。

```
class ECPoint:
    def __init__(self, x: int, y: int):
        self.x = x
        self.y = y
```

椭圆曲线参数

SM2 标准推荐参数（256 位素数域）：

p: 素数域模数

a, b: 曲线方程系数

n: 基点阶数（私钥取值范围）

G: 基点（生成元）

```
p = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF00000000FFFFFFFFFFFFFFFF
G = ECPoint(0x32C4AE2C..., 0xBC3736A2...) # 简化显示
```

核心运算函数

extended_gcd(): 扩展欧几里得算法，用于模逆计算

mod_inv(): 安全计算模逆（避免除零错误）。

ec_add(): 点加法（含点加倍处理）。

ec_mul(): 高效标量乘法（二进制展开优化）。

二、SM2 核心功能实现

密钥生成

生成私钥 d （随机数）和公钥 $P = d * G$ ：

```
def sm2_keygen() -> Tuple[int, ECPoint]:
    d = random.randint(1, n - 1)
    P = ec_mul(d, G) # 公钥为椭圆曲线点
    return d, P
```

ZA 计算

用户标识哈希值，包含曲线参数和公钥信息，确保身份绑定：

```
def calculate_zs(user_id: str, pub_key: ECPoint) -> bytes:
    ent1 = len(user_id) * 8
    data = f"{ent1:04X}{user_id}{a:064X}{b:064X}..."
    return sm3_hash(data) # 使用SM3哈希（代码中用SHA-256简化）
```

签名流程

1. 计算 ZA 并生成消息摘要 $e = \text{Hash}(ZA \parallel \text{msg})$ 。
2. 生成随机数 k ，计算点 $P_k = k * G$ 。
3. 计算 $r = (e + P_k.x) \bmod n$ ，若 $r=0$ 或 $r+k=n$ 则重试。
4. 计算 $s = (1+d)^{-1} * (k - r*d) \bmod n$ 。

```
r = (e + P_k.x) % n
s = mod_inv(1 + d, n) * (k_val - r * d) % n
```

验签流程

1. 检查 r, s 范围是否合法。
2. 计算 $t = (r + s) \bmod n$ ，若 $t=0$ 则失败。
3. 计算点 $P = s * G + t * \text{pub_key}$ 。
4. 验证 $(e + P.x) \bmod n == r$ 。

```
P = ec_add(ec_mul(s, G), ec_mul(t, pub_key))
R = (e + P.x) % n
return R == r
```

三、poc 验证

poc1_leaking_k()
poc2_reusing_k()
poc3_two_users_same_k()
poc4_malleability()
poc5_forge_without_m_check()
poc6_same_dk_ecdsa_sm2()

分别实现了 6 种 poc 验证

四、运行结果

==== SM2签名误用POC验证 ====

==== POC 1: 泄露k导致私钥泄露 ====

原始私钥: 16157387885063800092468972531095442600227637936690303362357377535130907802014
恢复私钥: 16157387885063800092468972531095442600227637936690303362357377535130907802014
恢复是否成功? True

==== POC 2: 重用k导致私钥泄露 ====

原始私钥: 69709006495262083753438964270882567809667203355268795714903518762464260067738
恢复私钥: 69709006495262083753438964270882567809667203355268795714903518762464260067738
恢复是否成功? True

==== POC 3: 两个用户使用相同k导致私钥泄露 ====

用户B的原始私钥: 24931630262289254097214708091063276378844549758072156217765307150175356726206
用户A恢复的私钥: 24931630262289254097214708091063276378844549758072156217765307150175356726206
恢复是否成功? True

==== POC 4: 签名可锻性 ====

原始签名 (r, s): (34510811647041748735505362440108717904547038460459720679741395656680156157700,
42524684437001910160212255274346490596216842954085339532902706575186616577344)
原始签名有效: True

==== POC 5: 不检查消息导致签名伪造 ====

伪造签名: (r=114080858501358080113819809317652355084681444685010988523266983451274686419441,
s=52501326164280954343698450070125624996742870340154584073996146161991243611954)
安全风险: 系统接受任何有效的(r,s)对作为有效签名
伪造签名'验证'结果: 取决于系统实现

==== POC 6: SM2和ECDSA使用相同d和k ====

原始私钥: 77600983479337250455748474695464935596938250988754697229835383426169547011888
恢复私钥: 77600983479337250455748474695464935596938250988754697229835383426169547011888
恢复是否成功? True

==== 所有POC验证完成 ====

根据运行结果, 所有 Poc 验证均执行成功。

三、中本聪数字签名伪造

一、代码结构与核心组件

开头导入 random 和 hashlib 模块，用于生成随机数和计算哈希值。

```
class EllipticCurve:
    def __init__(self, A, B, P, N, GX, GY):
        self.A = A
        self.B = B
        self.P = P
        self.N = N
        self.G = (GX, GY)
```

EllipticCurve 类实现椭圆曲线密码学（ECC）的核心运算

inv_mod 方法通过扩展欧几里得算法计算模逆元，解决模运算下的除法问题。

point_addition 实现点加法，处理两点相异或相同（切线）的情况，通过斜率计算新点坐标。

point_multiply 通过倍点算法高效实现标量乘法（如计算 k 倍点），将 k 转为二进制逐位处理。

```
# 比特币使用的secp256k1曲线参数
P = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEFFFFC2F
A = 0
B = 7
N = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141
GX = 0x79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798
GY = 0x483ADA7726A3C4655DA4FBFC0E1108A8FD17B448A68554199C47D08FFB10D4B8
```

比特币 secp256k1 曲线参数定义比特币使用的椭圆曲线标准参数

P=0xFFFFFFFF...FFFC2F：256 位素数，定义有限域大小。

A=0, B=7：曲线方程 $y^2 = x^3 + 7$ 的系数。

N=0xFFFF...D0364141：基点 G 的阶，为质数。

GX, GY：基点 G 的坐标，用于生成公钥。

```
class SatoshiForgery:
    def __init__(self):
        self.curve = EllipticCurve(A, B, P, N, GX, GY)
        self.G = (GX, GY)
        # 中本聪创世区块地址 (1A1zP1eP5QGeFi2DMPTfTL5SLmv7DivfNa)
        self.satoshi_pubkey = (0x11DB93E1DCDB8A016B49840F8C53BC1EB68A382E97B1482ECAD7B148A6909A5C,
                                0x0C6F5D7A825D1A0B3EF5E3CC3BDA8F0A7269C8C17272E9F13D6BDC0F10EEE8E)

    def sm3_hash(self, message):
        """ 简化版哈希函数 (实际应使用完整SM3) """
        h = hashlib.sha256()
        h.update(message.encode('utf-8'))
        return int.from_bytes(h.digest(), 'big') % N
```

中本聪公钥与类初始化 SatoshiForgery 类初始化时：

加载 secp256k1 曲线对象。

设置中本聪公钥(0x11DB93..., 0x0C6F5D...), 对应比特币创世区块地址 1A1zP1...DivfNa。

```
class SatoshiForgery:
    def __init__(self):
        self.curve = EllipticCurve(A, B, P, N, GX, GY)
        self.G = (GX, GY)
        # 中本聪创世区块地址 (1A1zP1eP5QGeFi2DMPTfTL5SLmv7DivfNa)
        self.satoshi_pubkey = (0x11DB93E1DCDB8A016B49840F8C53BC1EB68A382E97B1482ECAD7B148A6909A5C,
                                0x0C6F5D7A825D1A0B3EF5E3CC3BDA8F0A7269C8C17272E9F13D6BDC0F10EEE8E)

    def sm3_hash(self, message):
        h = hashlib.sha256()
        h.update(message.encode('utf-8'))
        return int.from_bytes(h.digest(), 'big') % N
```

sm3_hash 方法模拟哈希函数（实际应为 SM3），用 SHA-256 计算消息摘要并取模 N。

```
def forge_signature(self, message):
    """ 伪造中本聪签名核心方法 """
    # 步骤1: 生成随机数a和b
    a = random.randint(1, N - 1)
    b = random.randint(1, N - 1)

    # 步骤2: 计算 R = a*G + b*P
    aG = self.curve.point_multiply(a, self.G)
    bP = self.curve.point_multiply(b, self.satoshi_pubkey)
    R = self.curve.point_addition(aG, bP)
    R_x, _ = R

    # 步骤3: 计算伪造的签名参数
    r = R_x % N
    s = (r * self.curve.inv_mod(b, N)) % N
    e = (r * a * self.curve.inv_mod(b, N)) % N

    # 步骤4: 构造可验证的签名
    forged_signature = (r, s)
    return forged_signature, e
```

伪造签名核心算法 forge_signature 方法分四步伪造签名：

步骤 1: 生成随机数 $a, b \in [1, N-1]$ 作为临时私钥。

步骤 2: 计算点 $R = a \cdot G + b \cdot P$, 其中 P 为中本聪公钥。通过点乘和点加实现。

步骤 3: 取 R 的 x 坐标得 r , 计算 $s = r/b \bmod N$ 和伪造的哈希 $e = r \cdot a/b \bmod N$ 。

步骤 4: 返回伪造的签名 (r, s) 和哈希 e 。此构造使 (r, s) 对 e 通过验证。

签名验证逻辑 `verify_forged_signature` 模拟 ECDSA 验证流程:

计算 $w = s^{-1} \bmod N$ 。

推导 $u1 = e \cdot w, u2 = r \cdot w$ 。

计算点 $X = u1 \cdot G + u2 \cdot P$, 若 X 的 x 坐标模 N 等于 r 则验证通过。此过程依赖椭圆曲线数学性质, 与真实签名验证一致。

演示流程与攻击原理 `demonstrate_forgery` 方法:

伪造声明消息 (如承认 Craig Wright 为中本聪)。

生成并打印伪造的 r, s, e (64 位十六进制格式)。

```
def verify_forged_signature(self, signature, e):
    """验证伪造的签名"""
    r, s = signature
    w = self.curve.inv_mod(s, N)
    u1 = (e * w) % N
    u2 = (r * w) % N

    u1G = self.curve.point_multiply(u1, self.G)
    u2P = self.curve.point_multiply(u2, self.satoshi_pubkey)
    x, _ = self.curve.point_addition(u1G, u2P)

    return r == x % N
```

调用验证逻辑并输出结果。

攻击原理: 利用 ECDSA 验证时接受外部哈希的漏洞。通过控制 a, b 构造特殊点 R , 使 (r, s) 对攻击者指定的 e 有效。此漏洞曾被用于伪造身份证明。

执行与输出主程序创建 `SatoshiForgery` 对象并运行演示:

打印伪造声明和签名参数。

显示验证结果 (成功时警告)。

输出攻击原理说明, 强调其教育意义 (实际中本聪私钥未知)

二、核心漏洞原理

伪造签名的数学基础:

构造: $R = uG + vQ$

令: $r = R_x \bmod n$ 、 $s = rv^{-1} \bmod n$ 、 $e = us \bmod n$

验证时会发生:

$$\begin{aligned} P &= \left(\frac{e}{s}\right) G + \left(\frac{r}{s}\right) Q \\ &= \left(u \cdot \frac{s}{s}\right) G + \left(\frac{r}{s}\right) \cdot (d \cdot G) \\ &= u \cdot G + \left(\frac{r}{s}\right) \cdot d \cdot G \\ &= R \end{aligned}$$

三、运行结果

```
=====
中本聪签名伪造演示
=====
伪造声明: 「Craig Wright是比特币的真正创造者」

伪造签名生成结果:
r = 0x8bd767bfb01f2e9d779729baffc1b1e954472f3fbecce700281094e2399bb6fc
s = 0xa83b812cba2b0df7d373f5ffa7a7bd205afde53c7cdef3f7df161e0528806a18
消息哈希 = 0x64431bbe6dd9f73894399deef69fc9c798d09d72a3659c84167954410662decb

签名验证结果: 成功
伪造签名通过中本聪公钥验证
```

可以看到, 代码成功伪造了中本聪的数字签名, 伪造的签名通过了公钥验证。